



Escuela de Ingeniería Informática

Escuela de Ingeniería Informática
School of Computer Science Engineering

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Sistemas Distribuidos e Internet

Parte 2

Introducción a Node.js

Desarrollo de aplicaciones y servicios web
modernos con JavaScript



Dr. Edward Rolando Núñez Valdez

nunezedward@uniovi.es

Índice

- Arquitectura y módulos
- Bases de datos MongoDB
 - Servidor de bases de datos MongoDB
 - Conexión a bases de datos MongoDB
 - Gestión de datos con MongoDB
 - Arquitectura: acceso a datos
 - Insert
 - Remove
 - Update
 - Find
 - ObjectID
- Encriptación (Cifrado)
- Autenticación y autorización
 - Autenticación
 - Sesión y autorización
 - Enrutadores
- Subida de ficheros
- Captura de errores
- Https

Arquitectura y módulos

- La aplicación Node.js debería seguir una arquitectura modular:
 - Dividendo las responsabilidades en diferentes módulos.
 - Estos módulos pueden comunicarse entre sí.
- Comparándolo con Spring podríamos decir que:
 - Los módulos que definen endpoints o URLs actúan como controladores.
 - Los módulos que definen lógica de negocio actúan como servicios.
 - Los módulos de acceso a datos actúan como repositorios.

Arquitectura y módulos

- Por sus características Node.js es un muy buen candidato para **entornos muy dinámicos y desarrollos ágiles**
 - Aplicaciones con requerimientos que se modifican frecuentemente o han sido poco definidos.
 - Los cambios deben realizarse de forma rápida y efectiva (modificando poco código).
- En un desarrollo rápido el **tamaño de la aplicación puede condicionar su arquitectura**
 - Algunas aplicaciones con poca lógica de negocio pueden incluso prescindir de la capa de servicios (sí estos servicios realizan básicamente llamadas a repositorios)

Arquitectura y módulos

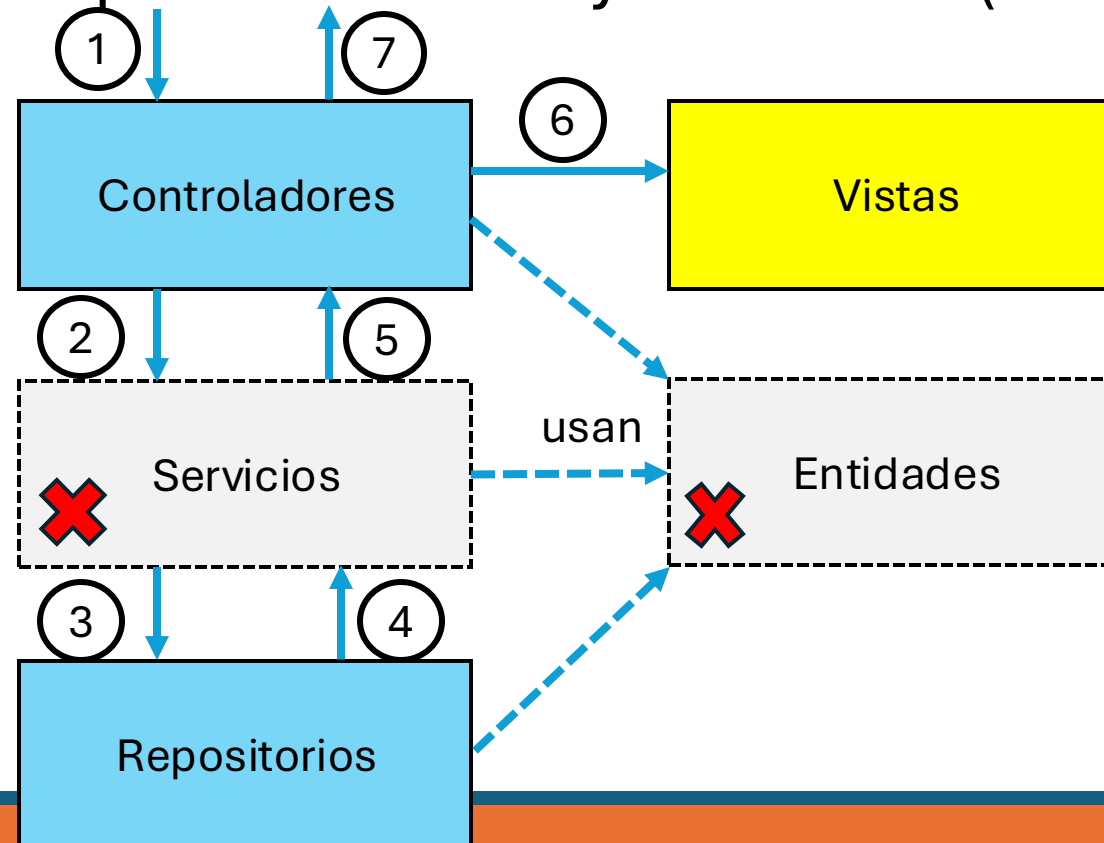
- Muchas aplicaciones no formalizan las entidades de forma estricta (con clases)
 - Una alternativa es **usar objetos**, donde resulta muy rápido modificar/añadir campos
 - Por ejemplo: objeto genérico canción (song) con 3 atributos

```
var song = {  
    title : req.body. title,  
    kind  : req.body. kind,  
    price : req.body.price  
}
```

- Los objetos se pueden utilizar de forma directa en muchas bases de datos no relacionales.

Arquitectura y módulos

- La arquitectura de una aplicación pequeña y dinámica podría prescindir de la capa de servicios y entidades (clases)



Recordando... Arquitectura y módulos >

¿Sistemas de módulos en Node.js?

- Los **módulos** permiten dividir el código de una aplicación en partes **reutilizables y organizadas**.
- Node.js soporta dos sistemas de módulos:
 - **CommonJS (CJS)**
 - Sistema tradicional de Node.js.
 - Utiliza **require()** para importar módulos y **module.exports** para exportarlos.
 - **ESM (ECMAScript Modules)**
 - Sistema moderno basado en el estándar de JavaScript.
 - Utiliza **import** para importar y **export** exportar módulos

```
module.exports = function (app) {...}  
  
let usersRouter = require('./routes/users');
```

```
// exportar el módulo de usuarios  
const usersRouter = (app) => {...};  
export default usersRouter;  
  
import usersRouter from './routes/users.js';
```



Arquitectura y módulos > ¿Qué puede exportar un módulo?

- Un **módulo es un archivo** que puede exportar diferentes elementos (funcionalidades) que pueden ser utilizadas por otros archivos.
- Los módulos se pueden exportar:
 - Funciones
 - Objetos
 - Clases

Arquitectura y módulos > ¿Qué puede exportar un módulo?

- Módulo exportado como ***una función***

```
module.exports = function(app, twig) {  
  const limits = 100;  
  app.get("/publications", function(req, res) {  
    ...  
  });  
  app.get('/shoping', function (req, res) {  
    ...  
  })  
}
```

Declaración del módulo /routes/rsongs.js

```
require("../routes/rsongs.js")(app, twig);
```

Importar el módulo /routes/rsongs.js

Arquitectura y módulos > ¿Qué puede exportar un módulo?

- Los **módulos exportados como objetos** permiten acceder a sus atributos y métodos utilizando el operador punto (.).
- Dentro del propio objeto, sus propiedades y métodos pueden referenciarse mediante la palabra clave: **this**

```
module.exports = {  
  name : null,  
  lastname : null,  
  init : function(name) {  
    this.name = name;  
  },  
  greeting : function(personalized) {  
    if (personalized == true )  
      return "Hola " + this.name;  
    else  
      return "Hola";  
  }  
};
```

init nombre común
para el inicializador

this.name es la variable
nombre del objeto

```
var person = require("../modules/person.js");  
person.init("John");
```

Declaración del módulo /modules/person.js

Importar el módulo /modules/person.js

Arquitectura y módulos > ¿Qué puede exportar un módulo?

- La sintaxis de un objeto es muy distinta a la de una función y a una clase

```
var objeto = {  
  <nombre_atributo> : <valor>,  
  <nombre_atributo> : <valor>,  
  <nombre_atributo> : <valor>,  
  <nombre_función> : function( <parámetros> ){  
  
    },  
  <nombre_función> : function( <parámetros> ){  
  
    }  
};
```

- **Un objeto no es lo mismo que una clase**
- require(<path del módulo>) siempre retorna la **misma instancia de objeto**
 - Aunque se importe varias veces todos retornan la referencia al mismo objeto

Arquitectura y módulos > ¿Qué puede exportar un módulo?

- Los **módulos exportados como una clase** permiten crear instancias .
- Dentro del propio objeto, sus propiedades y métodos pueden referenciarse mediante la palabra clave: **this**

```
//Usando CommonJS
module.exports = class Person {
  constructor(name) {
    this.name = name;
  }
  greeting (personalized) {
    if (personalized == true )
      return "Hola " + this.name;
    else
      return "Hola";
  }
};
```

Clase Persona

Constructor

```
var Person=require("./modules/person.js");
var person1 = new Person("John");
var person2 = new Person("J");
```

**2 instancias de
Persona**

```
// Usando ES Modules (ESM)
export default class Person {...};
```

Arquitectura y módulos > ¿Sistemas de módulos en Node.js?

- Módulos con múltiples exportaciones
 - Si el módulo tiene varios submódulos, funciones o variables, se pueden importar de forma individual

```
const {check} = require('express-validator');
exports.songValidatorInsert = [
  check('title', 'Title is required').trim().not().isEmpty(),
]

exports.songValidatorUpdate = [
  check('title')
    .if(check('title').exists())
    .trim().isLength({min: 5})
    .withMessage('Title must be 5 or more characters')
]
```

songsValidator.js

```
const {songValidatorInsert} = require('./songsValidator')
```

songsAPI.js

Arquitectura y módulos > Tipos de módulos

- Módulos externos:
 - Son módulos **descargados normalmente mediante NPM** y proporcionan funcionalidades reutilizables para muchas aplicaciones
 - Ejemplos:
 - body-parser
 - mongodb
 - twig
 - crypto
 - etc.
- Módulos propios
 - Son módulos desarrollados dentro de la propia aplicación y suelen **implementar funcionalidades específicas** del proyecto
 - Ejemplo:
 - songsManager
 - songsRoutes
 - etc.

Arquitectura y módulos > Uso de los módulos en la aplicación

- Los módulos pueden utilizarse en diferentes partes de la aplicación:
 - Desde la aplicación Node.js/Express :
 - Ejemplo:
 - El módulo **body-parser** es utilizado por la aplicación para procesar los datos enviados en el **body de las peticiones POST**.
 - Desde otros módulos:
 - Un módulo también puede utilizar **otros módulos**
 - Ejemplo
 - El módulo **songsRouter** utiliza:
 - **songsManager** para acceder a las canciones.
 - **twig** para generar respuestas basadas en plantillas.

Arquitectura y módulos > Uso de los módulos en la aplicación

- Alternativas de uso de módulos (externos o propios) desde otros módulos o partes de la app:

- Caso 1: Importar el objeto/función allí donde sea requerido.

- Alternativa no muy mantenible, los cambios pueden ser costosos)

```
util = require("utils.js");
```

users.js

```
util = require("utils.js");
```

payments.js

- Caso 2: Importar el objeto/función una vez y enviarlo como parámetro a otros módulos.

```
util = require("utils.js");
```

app.js

```
module.exports = function(util)
```

users.js

```
module.exports = function(util)
```

payments.js

- Caso 3: importar el objeto/función y almacenarlo en variables de la app

- Será, accesibles desde cualquier parte
 - No conviene abusar de las variables de app

```
app.set('util', require("utils.js"));
```

```
app.get('util');
```

users.js

```
app.get('util');
```

payments.js

Arquitectura y módulos > Uso de módulos de Express

- Usos de módulos integrados con la aplicación Express / externos
 - Primero se importa el objeto/función correspondiente al módulo
 - Luego se registra como un middleware en la app con `app.use(<objeto/funcion>)`
 - Ejemplos:
 - La app ya puede procesar cuerpos de peticiones (parámetros POST)

```
var bodyParser = require('body-parser');
app.use(bodyParser.json());
```
 - La app ya puede recibir ficheros en peticiones

```
var fileUpload = require('express-fileupload');
app.use(fileUpload());
```
- Los módulos otorgan nuevas funcionalidades a la app (muchas veces transparentes, no se requiere referenciar al módulo específico para obtener la funcionalidad)

Bases de datos MongoDB

Bases de datos MongoDB

- Son bases de datos no relacionales (NoSQL)
 - No existen tablas ni estructuras fijas que deban cumplir los datos almacenados
- ***Orientadas a documentos, donde la información se almacena en formato BSON***
 - BSON es una versión ligera creada a partir de JSON
 - <https://www.mongodb.com/json-and-bson>
- Un documento contiene un objeto BSON, con atributos que pueden tomar diferentes valores (tipos simples, objetos, colecciones, etc.)

Ventajas de MongoDB

- Sin esquema previo (Schema Less)
 - Permite almacenar datos no estructurados
- Flexible
 - Admite diferentes tipos de datos y estructuras en una misma colección.
- Escalable y distribuida
 - Soporta escalado horizontal y vertical
- Alta disponibilidad
 - Replica los datos entre nodos para garantizar tolerancia a fallos.
- Consultas complejas con sintaxis sencilla
 - Permite realizar consultas complejas con una sintaxis intuitiva basada en JSON.
- Código abierto
 - Software libre con una gran comunidad y soporte empresarial opcional.

Desventajas de MongoDB

- **Modelo no relacional**
 - No utiliza tablas ni claves foráneas como las bases de datos relacionales.
- **Gestión de relaciones más compleja**
 - Las relaciones se implementan mediante **documentos embebidos o referencias**.
- **Transacciones complejas**
 - Aunque MongoDB soporta **transacciones ACID**, su diseño está optimizado para operaciones sobre documentos.
 - **Atomicity** (Atomicidad)
 - **Consistency** (Consistencia / Integridad)
 - **Isolation** (Aislamiento)
 - **Durability** (Durabilidad / Persistencia)
- **Operaciones tipo join**
 - MongoDB permite joins mediante \$lookup, pero no están tan optimizados como en bases de datos relacionales.

MongoDB vs Bases de Datos Relacionales

Característica	MongoDB (NoSQL)	Bases de datos relacionales (SQL)
Modelo de datos	Documentos (JSON / BSON)	Tablas y filas
Esquema	Flexible (schema-less)	Esquema fijo
Relaciones	Referencias o documentos embebidos	Relaciones mediante claves foráneas
Consultas	Basadas en documentos	Lenguaje SQL
Joins	Posibles con \$lookup	Nativos y optimizados
Transacciones	Soportadas, pero menos utilizadas	Amplio soporte para transacciones complejas
Escalabilidad	Escalado horizontal sencillo	Escalado horizontal más complejo
Casos de uso	Aplicaciones web, big data, microservicios	Sistemas empresariales, aplicaciones financieras

¿Cuándo usar MongoDB?

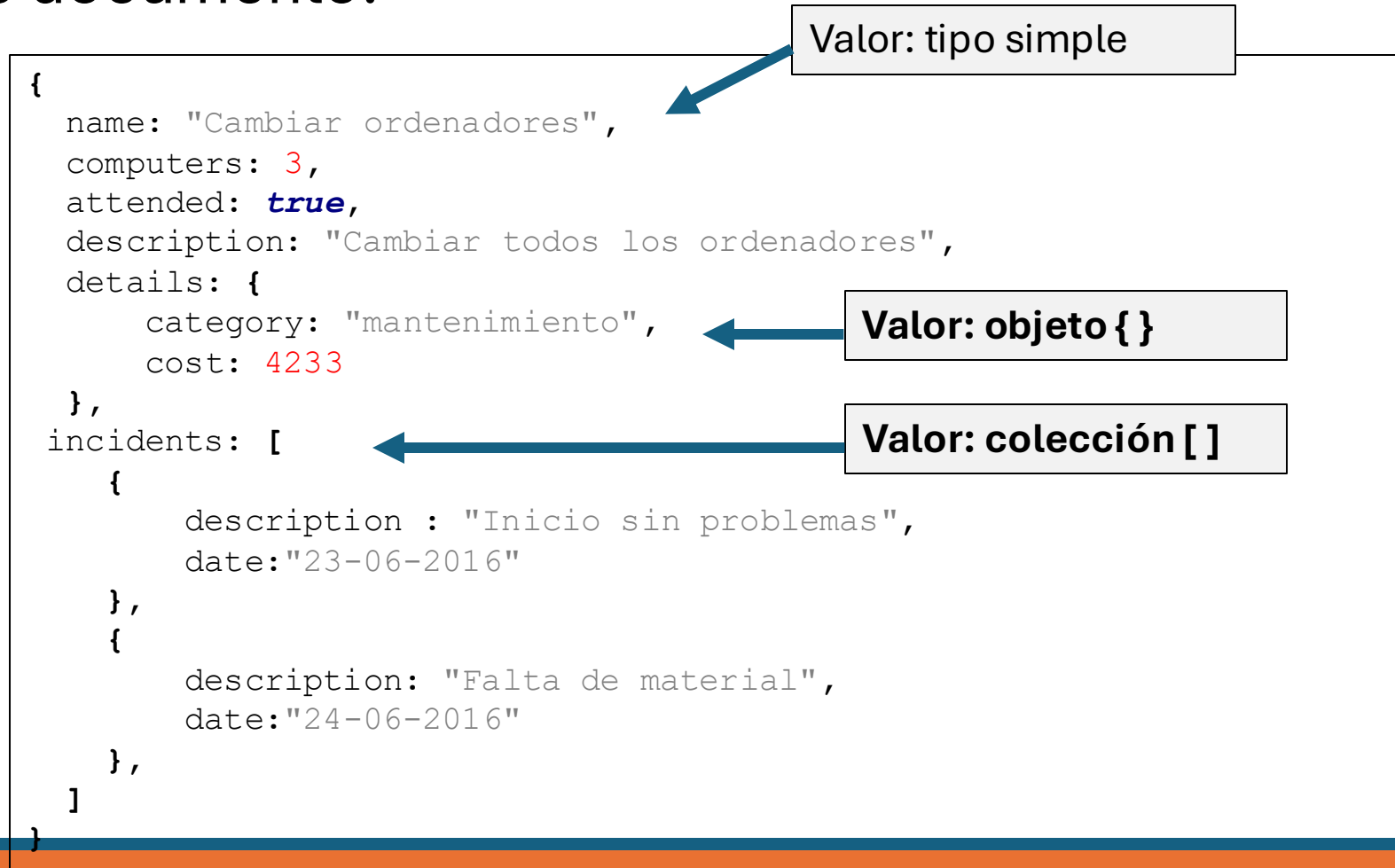
- MongoDB es especialmente adecuado cuando:
 - el modelo de datos es **flexible**
 - se trabaja con **grandes volúmenes de datos**
 - se requiere **escalabilidad horizontal**
 - los datos se representan naturalmente como **documentos**

¿Cuándo usar bases de datos relacionales?

- Las bases de datos relacionales son más adecuadas cuando:
 - existen **muchas relaciones entre entidades**
 - se requieren **transacciones complejas**
 - se necesita **alta integridad de datos**

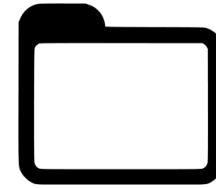
Bases de datos MongoDB

- Ejemplo documento:



Bases de datos MongoDB

- Los **documentos** se almacenan en **colecciones**
- Una **colección** es como una carpeta que agrupa documentos relacionados.
 - *(No es equivalente a una tabla relacional.)*
- La colección **no define la estructura** de los documentos
- Cada documento puede tener una **estructura diferente** (schema-less).
- La estructura de un documento puede **modificarse dinámicamente** según las necesidades de la aplicación.



Colección proveedores

```
{  
  "name" : "John",  
  "lastname" : "Doe",  
  "quality" : 10  
}
```

```
{  
  "center" : "uniovi",  
  "quality" : 10  
}
```

Bases de datos MongoDB

- Las **colecciones** permiten gestionar los documentos mediante operaciones como:
 - **Colección.find**({ criterio de selección }): Devuelve todos los documentos que cumplen con el criterio.
 - **Collección.insertOne**({ documento}): insertar un nuevo documento
 - **Colección.updateOne**({criterio de selección }, { nuevo documento }): actualizar documentos
 - **Colección.deleteOne**({criterio de selección }): eliminar documento
 - Otros.

Bases de datos MongoDB

- Al guardar un documento, MongoDB agrega automáticamente el campo:
 - `_id : ObjectId(...)`
- **ObjectId** actúa como **identificador único** del documento (clave primaria).
- **Se genera automáticamente**, aunque también puedes asignarlo manualmente si lo necesitas.
- Estructura del ObjectId (12 Bytes)
 - 4 bytes: Timestamp, momento de creación
 - 3 bytes: Identificador de la máquina
 - 2 bytes: PID – ID del proceso que lo generó
 - 3 bytes: Contador incremental

```
> db.proyectos.find()  
{ "_id" : ObjectId("574449da40fb278c24332fa6"), "nombre"  
  "opcion" : "Cambiar todos los ordenadores" }
```

Formas de trabajar con MongoDB

- **En Local**

- Instalación de MongoDB en un ordenador (Windows, Linux, macOS).
- Requiere configurar el servidor MongoDB localmente.
- Herramientas recomendadas:
 - MongoDB Compass (GUI para visualizar datos).
- **Ventajas:**
 - Control total del entorno.
 - Sin necesidad de conexión a internet.
 - Ideal para desarrollo y pruebas locales.

Formas de trabajar con MongoDB

- **En la Nube**

- Uso de MongoDB Atlas (servicio oficial en la nube).
- Proveedor de base de datos como servicio (DBaaS).
- Permite crear clústeres, backups automáticos, escalado y alta disponibilidad.

- **Ventajas**

- Acceso remoto desde cualquier lugar.
- No requiere instalación ni configuración compleja.
- Fácil escalabilidad y monitoreo.

Servidor de bases de datos MongoDB

- **Instalación Local** descargando el servidor de la página oficial <https://www.mongodb.com/es>
 - Ejecutar el instalable y completar la instalación
 - Crear la **carpeta** para almacenar las bases de datos, por ejemplo: **C:\data\db**
 - Acceder a la carpeta donde se instaló **\MongoDB\Server\3.4\bin** y ejecutar el comando de arranque del servidor:
mongod --dbpath C:\data\db

```
2016-10-16T18:00:04.543+0200 I - [initandlisten] Detected data files in C:\data\db\ crea
storage engine, so setting the active storage engine to 'wiredTiger'.
2016-10-16T18:00:04.544+0200 I STORAGE [initandlisten] wiredtiger_open config: create,cache_si
viction=(threads_max=4),config_base=false,statistics=(fast),log=(enabled=true,archive=true,path
y),file_manager=(close_idle_time=100000),checkpoint=(wait=60,log_size=2GB),statistics_log=(wait
2016-10-16T18:00:04.716+0200 I NETWORK [HostnameCanonicalizationWorker] Starting hostname cano
2016-10-16T18:00:04.716+0200 I FTDC [initandlisten] Initializing full-time diagnostic data
:/data/db/diagnostic.data'
2016-10-16T18:00:04.718+0200 I NETWORK [initandlisten] waiting for connections on port 27017
```

- La cadena de conexión será: **mongodb://localhost:27017/<nombre base de datos>** (Por defecto acceso libre sin usuario)
 - Sí la base de datos no existe, se crea al conectarse

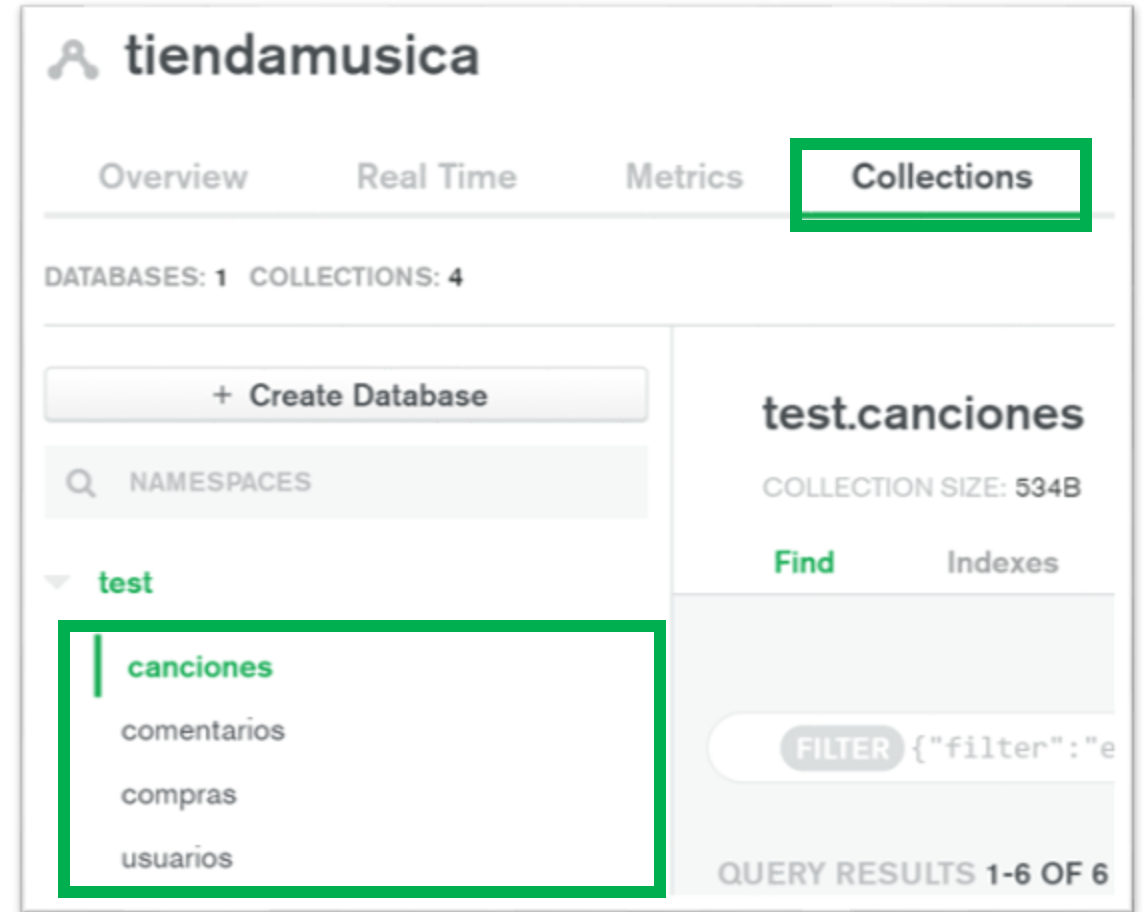
Servidor de bases de datos MongoDB

- Mongo en la Nube usando un proveedor de cloud computing
 - **MongoDB Atlas** (y otros muchos proveedores) permiten la creación de bases de datos en la Nube
 - <https://www.mongodb.com/>
 - Permite servidores “elásticos” pudiendo cambiar entre servidores con más o menos recursos según el uso requerido
 - Ofrece 512mb de almacenamiento de datos sin coste
 - Permite crear múltiples bases de datos
 - Por seguridad requiere la creación de un usuario-password para la base de datos
 - Obtenemos una cadena de conexión, por ejemplo:

<mongodb://<dbuser>:<dbpassword>@ds229008.mlab.com:29008/movil>

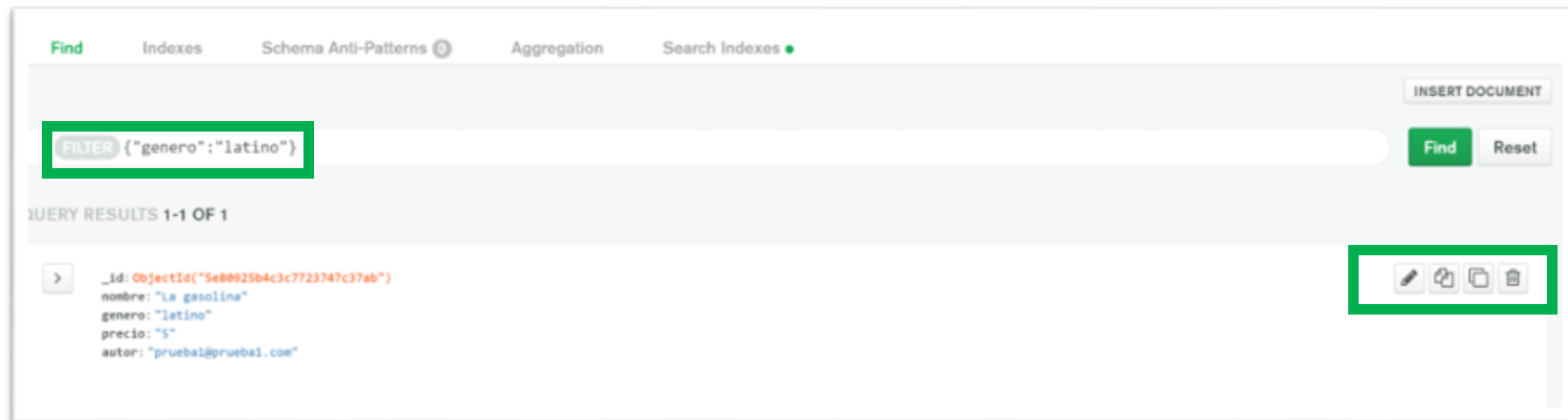
Servidor de bases de datos MongoDB

- Desde <https://cloud.mongodb.com/> puede consultar todas las **colecciones** y **documentos** que se van creando.
 - Ejemplos colecciones: **canciones** y **usuarios**...



Servidor de bases de datos MongoDB

- Se puede ver el **contenido de los documentos**, realizar ***búsquedas***, ***insertar***, ***borrar y modificar***.



Conexión a bases de datos MongoDB

- Para que una aplicación Node.js se conecte a una base de datos se requiere un **módulo** (librería)
- Cada motor de bases de datos utiliza un módulo propio
- El módulo **mongodb** es el driver oficial para MongoDB en Node.js
 - <https://mongodb.github.io/node-mongodb-native/>
 - No está incluido en el core de Node.js
- Actualmente se mantienen varias versiones release de **mongodb**: 6.X a 7.X cada una puede utilizar API muy diferente

RELEASE	DOCUMENTATION
Next Driver	Reference API
7.1 Driver	Reference API
7.0 Driver	Reference API
6.21 Driver	Reference API
6.20 Driver	Reference API
6.19 Driver	Reference API
6.18 Driver	Reference API
6.17 Driver	Reference API
6.16 Driver	Reference API
6.15 Driver	Reference API
6.14 Driver	Reference API
6.13 Driver	Reference API
6.12 Driver	Reference API
6.11 Driver	Reference API

Conexión a bases de datos MongoDB

- En la instalación del módulo se debe especificar la versión.

```
npm install mongodb@7.1
```



Especificar versión
concreta

- Si no especificamos versión instala la que “considera la última”
- Para importar mongodb a la aplicación:

```
let express = require('express');  
let app = express();  
const {MongoClient} = require("mongodb")
```

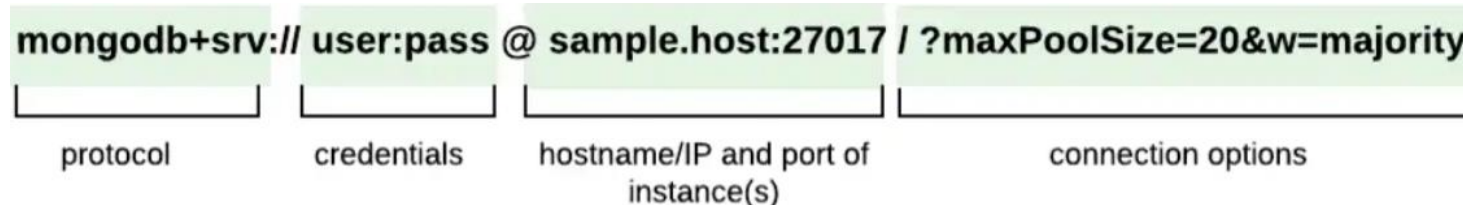
Conexión a bases de datos MongoDB

- El módulo **mongodb** contiene todo lo necesario para conectarnos a la base de datos
 - Incluido el cliente **mongo.MongoClient**
 - El módulo **mongo** se envía a los módulos que realicen el acceso a datos.

```
usersRepository.init(app, MongoClient);  
require("./routes/users.js")(app, usersRepository);
```



- Para conectarse a la base de datos debemos usar una **URL de conexión**:
 - Esta URL de conexión se puede guardar en una variable de entorno, fichero de configuración o en las **variables de la aplicación**
 - Esta última permitirá usarla en todos los módulos de la aplicación



```
app.set('connectionStrings', 'mongodb+srv://admin:<db_password>@musicstoreapp-cluster.jiclwfi.mongodb.net/?appName=musicstoreapp-cluster');
```

Conexión a bases de datos MongoDB

- El módulo **MongoClient** permite **conectarse** a una base de datos Mongo
 - La función **connect** requiere los parámetros:
 - URL de conexión
 - Función **manejadora (Handler)**, con dos parámetros:
 - **err** -> en caso de haber errores este parámetro toma valor, incluye el mensaje del error
 - **db** -> referencia a la base de datos, sobre este objeto se realizan las acciones (insertar, buscar, etc.)

```
mongo.MongoClient.connect(app.get('db'), function(err, db)
{
  if (err) {
    // Error al conectar
  } else {
    // usar "db" para realizar acciones (insertar, etc.)
  }
});
```

Ejemplo 1: Conexión con callback

```
try {
  const client = await this.mongoClient.connect(this.app.get('connectionStrings'));
  // realizar acciones (insertar, etc.)
  return result.insertedId;
} catch (error) {
  // Error al conectar
  throw (error);
}
```

Ejemplo 2: Conexión con Async/Await

Gestión de datos con MongoDB

- En la función de callback (ejemplo1) usaremos el objeto **db** para gestionar los datos
 - **db.collection**(<nombre colección>) da acceso a una colección
 - Se pueden referenciar incluso colecciones no existentes
 - Sí guardamos un documento en una colección no existente se creará la colección.
- Sobre la colección se realizan las acciones, por ejemplo:
 - **insertOne**(objeto JSON, función manejadora(err, resultado)) -> para guardar un nuevo documento
 - El resultado de la función manejadora depende de la acción:
 - Las inserciones retornan el documento insertado (con su `_id`)
 - Las búsquedas retornan listas de documentos
 - Etc.
- **Todas las acciones (al igual que la conexión) son asíncronas**
 - Cuando la acción se completa se invoca la función manejadora
 - Podemos usar callback, promesas y `async/await`

Recordando ... Operaciones asíncronas en Node.js

- Node.js permite ejecutar **operaciones de forma asíncrona**, es decir, sin bloquear la ejecución del programa mientras se espera el resultado.
- Esto es especialmente **útil para** operaciones como:
 - acceso a **archivos**
 - consultas a **bases de datos**
 - peticiones **HTTP**
 - comunicación con **servicios externos**
- En Node.js existen varias formas de gestionar operaciones asíncronas:
 - **Callbacks**
 - **Promesas (Promises)**
 - **Async / Await**

Formas de manejar operaciones asíncronas

- **Callbacks**

- Una **callback** es una función que se pasa como parámetro y se ejecuta cuando termina la operación.

```
managerDB.insertSong(song, function(id) {  
    if (id == null) {  
        res.send("Error al insertar ");  
    } else {  
        res.send("Agregada id: "+ id);  
    }  
});
```

- Problema: cuando hay muchas operaciones encadenadas puede aparecer el **callback hell**.

Formas de manejar operaciones asíncronas

- **Promesas**

- Las promesas representan el resultado futuro de una operación.
- Las promesas tienen **tres estados posibles** que representan la situación de una operación asíncrona:
 - **Pending (Pendiente)**: Es el estado inicial de una promesa.
 - La operación asíncrona aún *no ha finalizado*.
 - Todavía *no se conoce el resultado*.
 - **Fulfilled (Cumplida o Resuelta)**: La operación se ha *completado correctamente*
 - La promesa *devuelve un resultado*.
 - Este resultado se recibe normalmente mediante *.then()*
 - **Rejected (Rechazada)**: La operación *ha fallado*.
 - Se produce un error.
 - El error se maneja normalmente con *.catch*
- Ejemplo:

```
...
managerDB.insertSong(song)
  .then(result => console.log(result.insertedId))
  .catch(err => console.error(err));
```

Formas de manejar operaciones asíncronas

- **Async/Await**

- Simplifican el uso de promesas y permiten escribir código asíncrono con apariencia **sincrónica**.

```
async function (song) {  
  ...  
  const result = await collection.insertOne(song);  
  if (result.insertedId > 0) {  
    console.log(result.insertedId)  
  } else {  
    console.error("No se ha insertado el documento")  
  }  
  ...  
}
```

- Es la forma **más utilizada actualmente en Node.js..**

Ejecución asíncrona con callback - Node.js/mongoDB

- Ejemplo **insert** usando callback

```
var song = {
  name : req.body.name,
  gender : req.body.gender
}
mongo.MongoClient.connect(app.get('db'), function(err, db) {
  if (err) {
    res.send("Error de conexión: " + err);
  } else {
    var collection = db.collection('songs');
    collection.insertOne(song, function(err, result) {
      if (err) {
        res.send("Error al insertar " + err);
      } else {
        res.send("Agregada ");
      }
    });
    db.close();
  }
});
```

manejador

Una vez acabado recomendado
cerrar la conexión

Ejecución asíncrona con callback - Node.js/mongoDB

- Debemos tener muy claro el concepto de ejecución **asíncrona**
- El código se va ejecutando por **fases** usando callback

1º Fase

```
var song = {
  name : req.body.name,
  gender : req.body.gender
}

mongo.MongoClient.connect(app.get('db'), function(err, db) {
  if (err) {
    res.send("Error de conexión: " + err);
  } else {
    var collection = db.collection('songs');
    collection.insertOne(song, function(err, result) {
      if (err) {
        res.send("Error al insertar " + err);
      } else {
        res.send("Agregada ");
      }
    });
    db.close();
  }
});
```

Sí respondemos aquí res.send(. . .) Más código

Cuidado!, la ejecución
no es síncrona

Ejecución asíncrona con callback - Node.js/mongoDB

- Segunda fase ejecución **asíncrona**

2º Fase connect

```
var song = {
  name : req.body.name,
  gender : req.body.gender
}

mongo.MongoClient.connect(app.get('db'), function(err, db) {
  if (err) {
    res.send("Error de conexión: " + err);
  } else {
    var collection = db.collection('songs');
    collection.insertOne(song, function(err, result) {
      if (err) {
        res.send("Error al insertar " + err);
      } else {
        res.send("Agregada ");
      }
      db.close();
    });
  }
});

Más código
Más código
```

Ejecución asíncrona con callback - Node.js/mongoDB

- Tercera fase ejecución **asíncrona**

```
var song = {
  name : req.body.name,
  gender : req.body.gender
}
mongo.MongoClient.connect(app.get('db'), function(err, db) {
  if (err) {
    res.send("Error de conexión: " + err);
  } else {
    var collection = db.collection('songs');
    collection.insertOne(song, function(err, result) {
      if (err) {
        res.send("Error al insertar " + err);
      } else {
        res.send("Agregada ");
      }
      db.close();
    });
  }
});
Más código
Más código
```

3º Fase insert



Respuesta final
En caso de Éxito

Ejecución asíncrona con promesa - Node.js/mongoDB

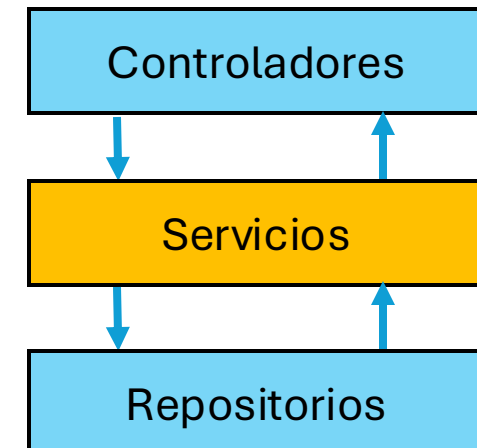
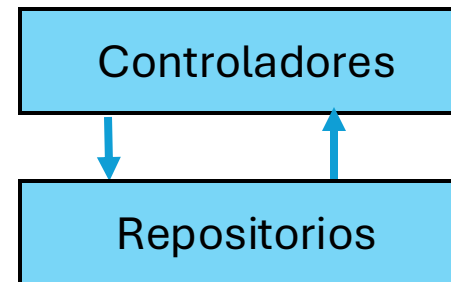
- Ejemplo **insertOne** usando promesas

```
...
insertSong: function (song, funcionCallback) {
  this.mongo.MongoClient.connect(this.app.get('db'), function (err, db) {
    if (err) {
      funcionCallback(null);
    } else {
      let collection = db.collection('songs');
      //Con promesa
      collection.insertOne(song)
        .then(result => funcionCallback(result.ops[0]._id))
        .catch(err => funcionCallback(null));
      db.close();
    }
  });
},
...

```


Arquitectura: acceso a datos

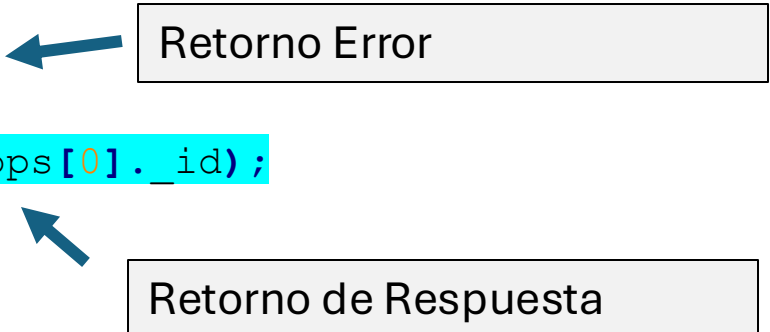
- Debemos **encapsular** el **acceso a datos en uno o varios módulos**
 - Dependiendo del número de **entidades y operaciones**, hay que valorar:
 - Un **único módulo para varias entidades** relacionadas
 - Un **módulo para cada entidad**
- Para **lógicas de negocio simples** los controladores podrían **utilizar los módulos de acceso a datos**
 - Por ejemplo: si la aplicación solo realiza operaciones CRUD básicas
- Para **lógica compleja** implementaríamos **módulos de servicios**



Arquitectura: acceso a datos

- Podemos definir un **módulo** como **objeto** que encapsule las operaciones
- Como las operaciones son **asíncronas** deben recibir una **función de callback** (retorno), **No usamos return**
 - **Función de callback:** se invoca al finalizar la operación asíncrona, por ejemplo: para enviarle el id del objeto insertado.

```
insertSong : function(song, funcionCallback) {  
    this.mongo.MongoClient.connect(this.app.get('db'), function(err, db) {  
        ...  
        var collection = db.collection('songs');  
        collection.insert(song, function(err, result) {  
            if (err) {  
                funcionCallback(null);  
            } else {  
                funcionCallback(result.ops[0]._id);  
            }  
            db.close();  
        });  
    });  
});
```



Retorno Error

Retorno de Respuesta

Arquitectura: acceso a datos

- Por ejemplo: uso de un módulo de acceso a datos desde un controlador
 - **managerDB** es la referencia al módulo

```
app.post("/song", function(req, res) {  
  var song = {  
    name : req.body.name,  
    genero : req.body.genero  
  }  
}
```

```
  managerDB.insertSong(song, function(id) {  
    if (id == null) {  
      res.send("Error al insertar ");  
    } else {  
      res.send("Agregada id: "+ id);  
    }  
  });  
});
```

Función de callback

Al acabar de **insertar**

Debe recibir la **id** de la canción insertada

```
funcionCallback(result.ops[0]._id);
```

Operaciones CRUD en MongoDB

Insertar un documento

- Método **InsertOne()** - Insertar un documento
 - Inserta un único documento en una colección.
 - Si el documento no contiene `_id`, MongoDB lo genera automáticamente.
 - Si la colección no existe, se crea automáticamente al hacer la inserción.
 - **Parámetros:**
 - **Documento**
 - Objeto que contiene los campos y valores que desea almacenar.
 - **options** (opcional).
 - Configuraciones como timeouts, serialización, etc..
 - **writeConcern.** define el nivel de confirmación de escritura.
 - Ejemplo: w:0, w:1, w:majority, w:tagSet
 - Puedes omitirlo para usar la configuración por defecto.
 - **callback** (opcional)
 - Función que se ejecuta al completar la operación (estilo asíncrono)
 - **Valor de retorno:**
 - El método devuelve un **objeto de resultado** que incluye:
 - **insertedId:** el `_id` del documento insertado.
 - **acknowledged:** booleano que indica si la operación fue exitosa.
 - w:0 -> false
 - w:1 o majority -> true

Insertar un documento

- Ejemplo insertar usando promesas

```
...
insertSong: function (song, funcionCallback) {
  this.mongo.MongoClient.connect(this.app.get('db'), function (err, db) {
    if (err) {
      funcionCallback(null);
    } else {
      let collection = db.collection('songs');
      //Con promesa
      collection.insertOne(song)
        .then(result => funcionCallback(result.insertedId))
        .catch(err => funcionCallback(null));
      db.close();
    }
  });
},
```

Colección

Documento a insertar

insertedId -> _id del documento insertado

Insertar varios documentos

- Método **InsertMany()** – Insertar varios documentos
 - Inserta un array de documentos.
 - Si los documentos no contienen `_id`, MongoDB los genera automáticamente.
 - Si la colección no existe, se crea al hacer la inserción
 - **Parámetros**
 - **Array de documentos**
 - Lista de objetos que deseas almacenar.
 - **options** (opcional).
 - **Configuraciones adicionales:** timeouts, serialización, etc.
 - **Ordered:**
 - `true` (por defecto): si un documento falla, detiene la inserción.
 - `false`: intenta insertar todos, aunque algunos fallen.
 - **writeConcern:**
 - Define el nivel de confirmación de escritura (ej. `w: 1`, `w: "majority"`).
 - Puedes omitirlo para usar el valor por defecto.
 - **Valor de retorno:** El método devuelve un objeto con los siguientes campos:
 - **insertedIds:** Lista de los `_id` generados o asignados a cada documento insertado.
 - **insertedCount:** Número de documentos insertados correctamente.
 - **acknowledged:** Booleano que indica si la operación fue confirmada.

Insertar varios documentos

- Ejemplo insertar usando Async/Await

```
...
Async function (song, funcionCallback) {
  this.mongo.MongoClient.connect(this.app.get('db'), function (err, db) {
    if (err) {
      funcionCallback(null);
    } else {
      const collection = db.collection('songs');
      //Con await
      const options = { ordered: true };
      const result = await collection.insertMany(songList, options)
      funcionCallback(result.insertedIds)
      db.close();
    }
  });
},
```

Colección

Documentos a insertar

insertedIds -> lista de _id de los documentos insertados

Borrar un documento

- Método **DeleteOne()** - Eliminar un documento
 - Elimina **el primer documento** que coincida con una consulta.
 - **Si no encuentra coincidencias, no elimina nada.**
 - **Si la colección no existe, no se realiza ninguna operación**
 - **Parámetros:**
 - **Consulta:**
 - Objeto que define los criterios de selección del documento a eliminar.
 - Ejemplos:
 - {"type": "casa"} = los documentos de tipo casa
 - {"price": { "\$gte": 31 }} = documentos con precio mayor o igual que 31
 - **Operadores lógicos:**
 - **\$gt** - greater than. Mayor que | **\$gte** - greater than or equal . Mayor o igual que | **\$lt** - less than, Menor que | **\$lte** - less than or equal. Menor o igual que
 - **{ \$or : [{"age" : 20}, {"age" : 30}, {"age" : 40}] }** = documentos con edad 20, 30 o 40 . OR
 - **{ \$and : [{"age" : 40}, {"empresa" : "CSC"}] }** = documentos con edad 40 y empresa CSC . AND
 - 1. **options (opcional).**
 - **Configuraciones adicionales:** timeouts, serialización, etc.
 - **writeConcern:**
 - Define el nivel de confirmación de escritura (ej. w: 1, w: "majority").
 - Puedes omitirlo para usar el valor por defecto.
 - Valor de retorno: Devuelve un objeto con los siguientes campos:
 - **acknowledged:** Booleano que indica si la operación fue confirmada.
 - **deletedCount:** Número de documentos eliminados (debe ser 1 o 0).
- Método **DeleteMany():** Elimina múltiples documentos
 - <https://docs.mongodb.com/drivers/node/current/usage-examples/deleteMany/>

Borrar un documento

- Ejemplo deleteOne()

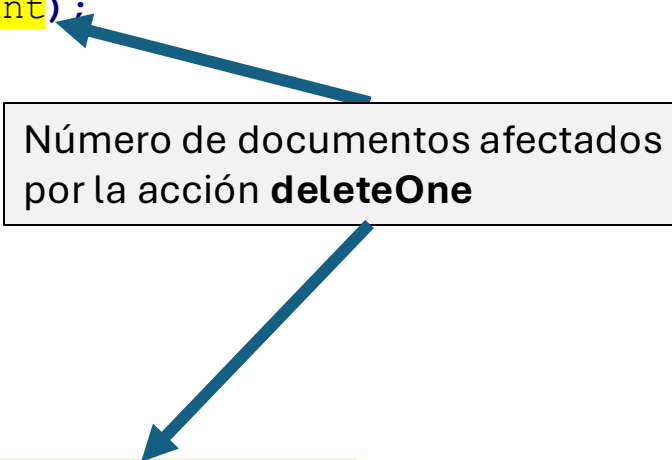
// Con Await

```
const query = {name: "James" };
const collection = db.collection('songs');
const result = await collection.deleteOne(query);
if (result.deletedCount === 1) {
  callbackControlador(result.deletedCount);
} else {
  callbackControlador(null);
}
db.close();
```

// Con promesas

```
const query = {name: "James" };
const collection = db.collection('songs');
collection.deleteOne(query)
  .then(result => callbackControlador(result.deletedCount))
  .catch(err => callbackControlador(null));
db.close();
};
```

Número de documentos afectados
por la acción **deleteOne**



Actualizar un documento

- Método **updateOne()** - Actualizar un documento
 - Actualiza **el primer documento** que coincida con un filtro de consulta.
 - Si **no encuentra coincidencias**, no actualiza nada.
 - Si la colección **no existe**, no se realiza ninguna operación
 - **Parámetros:**
 - **Filtro:** Objeto que define los criterios para seleccionar el documento a actualizar.
 - **Documento:** Objeto que contiene los cambios a realizar en el documento seleccionado
 - **Upsert:** Si se establece **upsert: true**, MongoDB **crea un nuevo documento si no se encuentran**
 - **Opciones (opcional).**
 - **Configuraciones adicionales** como timeouts, serialización, etc.
 - **writeConcern:** Define el nivel de confirmación de escritura (ej. w: 1, w: "majority").
 - **Valor de retorno:** Devuelve un objeto con los siguientes campos:
 - **acknowledged:** Booleano que indica si la operación fue confirmada.
 - **matchedCount:** Número de documentos que **coincidieron** con el filtro.
 - **modifiedCount:** Número de documentos **modificados**.
 - **upsertedCount:** Numero de documentos **insertados** debido a la operación upsert .
 - **upsertedId:** Identificador del documento insertado si se produjo una inserción (**upsert**).
 - Método **UpdateMany():** actualiza múltiples documentos de una colección
 - <https://docs.mongodb.com/drivers/node/current/usage-examples/updateMany/>

Actualizar un documento

- Ejemplo de actualización completa
 - Sustituye **completamente** el documento que cumple con el criterio de selección por el nuevo documento

```
const filter = { name : "James"};
const newPerson = { name : "R", lastName : "R"};
const options = { upsert: true } //crear un documento si no hay
documentos que coincidad con el filtro
const collection = db.collection('songs');
const result = await collection.updateOne(filter, newPerson,
options);
```

```
if (result.modifiedCount <= 0) {
  callbackController(null);
} else {
  callbackController(result);
}

db.close();
});
```

Pre-update

```
{
  "name": "J",
  "lastName": "J",
  "country": "es",
  "language": "es"
}
```

Post-update

```
{
  "name": "R",
  "lastName": "R"
}
```

Actualizar un documento

- Ejemplo actualización parcial **\$set**

- El operador **\$set** se utiliza para **sustituir o agregar atributos específicos** a un documento **sin modificar los demás campos**.
- Sintaxis básica: **{ \$set: objeto con atributos }**

```
var filter = { name : "J"};
var attributes = { lastName: "R", age : 40};
const options = { upsert: true }
const collection = db.collection('songs');
const result = await collection.updateOne(filter, { $set: attributes }, options);
if (result.modifiedCount <= 0) {
    callbackController(null);
} else {
    callbackController(result);
}
db.close();
});
```

Pre-update

```
{
  "name": "J",
  "lastName": "J",
  "country": "es",
  "language": "es"
}
```

Post-update

```
{
  "name": "J",
  "lastName": "R",
  "age": 40,
  "country": "es",
  "language": "es"
}
```

Buscar documentos

- Método **findOne()**: Buscar un documento
 - Se utiliza para **buscar un único documento** dentro de una colección que cumpla con un criterio de búsqueda específico
 - Si no proporciona un documento de consulta, se devolverá **el primer documento encontrado** en la colección.
 - **Parámetros:**
 - **Filtro:** Objeto que define los criterios para seleccionar el documento a buscar.
 - **Ejemplos**
 - {"type" : "casa"} = los documentos de tipo casa
 - {"type" : "casa", "metros" : 100} = los documentos de tipo casa y metros 100
 - Es equivalente a utilizar un AND
 - Se pueden utilizar operadores lógicos: \$gte, \$gt, \$lt, \$in, \$or, \$and \$in (valor contenido en un array), \$nin – valor NO contenido en un array, etc.
 - **Opciones (opcional).**
 - **Sort:** ordenar por un criterio
 - **Projection:** Incluir solo los campos especificado en el documento devuelto

Buscar documentos

- Método Find(): Obtener varios documentos
 - El método find() **devuelve un cursor** que permite realizar varias operaciones sobre los documentos que coinciden con la consulta.
 - Algunas de las operaciones más comunes son **toArray()**, **skip()** y **limit()**.
 - El método **toArray()** se utiliza para convertir el cursor que devuelve find() en un array de documentos.

```
const query = { title: "despacito" };
const options = { sort: { "title": 1 }, projection: { _id: 0, title : 1,
author : 1}};
const collection = db.collection('songs');
const cursor = collection.find(query, options);
const songs = await cursor.toArray();
if (songs.length <= 0) {
  callbackControlador(null);
} else {
  callbackControlador(songs);
}
db.close();
};
```

Buscar documentos

- Antes de obtener los documentos podemos aplicar **filtros**, usando los metodos:
 - **skip (número)**: Método usado para "**saltar**" un número determinado de documento.
 - **limit (número)**: Método se utiliza para **limitar el número de documentos** que se retornan en los resultados.
- Son útiles útil cuando deseas realizar paginación o empezar desde un punto específico en los resultados.
- Ejemplo para obtener 3 documentos

```
...  
const sort = { length: -1 };  
const limit = 3;  
const collection = db.collection('songs');  
const cursor = collection.find({}).sort(sort).limit(limit);  
const songs = await cursor.toArray();  
if (songs.length <= 0) {  
  callbackControlador(null);  
} else {  
  callbackControlador(songs);  
}  
...
```

Filtra el resultado antes de
obtener el array



ObjectID

- La transformación de objeto Mongo a JavaScript es **automática**

```
collection.find({}).toArray(function(err, usuarios) {  
    funcionCallback(usuarios);  
    db.close();  
});
```



Array de objetos. Cada objeto tiene los datos de **usuario**

- Los objetos JS recuperados de mongo tienen un **_id : ObjectId**
 - ObjectID** por defecto no es un tipo primitivo
 - El valor de este atributo es una **instancia de ObjectId**
 - En algunos casos, para acceder al valor como cadena: `<objeto>._id.toString()`
 - Ejemplo en JavaScript:

```
let usr = usuarios[0];  
let a = usr._id; // ObjectId  
let b = usr._id.toString(); // String
```

- Ejemplo en Swig o Twig:

```
<a href="/usr/{{ usr._id.toString() }}">
```

ObjectID

- Considerar ObjectID en los criterios de selección por **_id**
 - Tipo ObjectID no es Tipo String

```
app.get('/usuario/:id', function (req, res) {  
    var criterio = { "_id" : req.params.id };  
})
```



Los **_id** NO son de tipo String

- Posible solución: convertir el String recibido a ObjectId
 - El módulo **mongodb** permite crear ObjectIds, con la función: **mongo.ObjectId(String)**.

```
app.get('/usuario/:id', function (req, res) {  
    var objectID = new ObjectId(req.params.id);  
    var criterio = { "_id" : objectID };  
})
```

Encriptación (Cifrado) y Autenticación y autorización

Encriptación (Cifrado)

- El módulo **crypto** permite cifrar (encriptar) y descifrar (desencriptar)
 - <https://nodejs.org/api/crypto.html>
- Está incluido en el Core de Node.js (No hay que instalarlo)
 - El objeto crypto se obtiene con un require

```
var crypto = require('crypto');
```
- Es necesario cifrar las contraseñas y cualquier otra información que sea sensible
- Permite múltiples algoritmos de cifrado:
 - sha256, sha512, otros.
- Permite realiza múltiples codificaciones :
 - hex, latin1, base64, etc.

Encriptación (Cifrado)

- Requiere definir una “clave de cifrado” o “secreto”
 - **createHmac(<tipo>, <secreto>):** crea un objeto para realizar el cifrado

```
secreto = 'abcdefg';  
valor = "342434";  
encryptor = crypto.createHmac('sha256', secreto);
```

- **update(<valor a cifrar>):** retorna el valor cifrado
- **digest(<tipo>):** especifica la codificación del valor cifrado

```
encryptedValue = encryptor.update(valor).digest('hex');
```

Autenticación y autorización

- La **autenticación** consiste en **validar la identidad** de un usuario
- Como mínimo los usuarios se identifican usando:
 - **Username** : identificador único, ID, DNI, nombre, email, etc.
 - **Password**: contraseña del usuario
- Muchos frameworks proveen sistemas de autenticación/autorización
 - Estos sistemas siguen sus propios enfoques (diferentes entre ellos)
 - Son de muy alto nivel, suelen “abstraer” los conceptos
 - Ejemplo: Spring Security en Spring, Express-authentication en Express, etc.
- Implementar un sistema propio la alternativa a usar los provistos por los frameworks

Autenticación

- Un proceso de implementación de Autenticación podría ser:
 - 1. Definir un controlador que reciba la petición POST con los parámetros
 - username (en este caso email) y password

```
app.post("/login", function(req, res) {  
    var email = req.body.email;  
    var password = req.body.password;
```

- 2. Realizar una búsqueda en los usuarios por ambos criterios
 - En la base de datos el password está encriptado

```
    var seguro = encriptador.update(password).digest('hex');  
    var criterio = {  
        email : email,  
        password : seguro  
    }  
    dbManager getUsers(criterio, function(users)
```

← Password encriptado

← Array de usuarios que cumplen el criterio

Autenticación

- Un proceso de implementación de Autenticación podría ser:
 3. ¿Retorna algún usuario con ese criterio de búsqueda?
 - Null o 0 – **No se ha autenticado**, redireccionar a la URL apropiada
 - 1 usuario – **Se ha autenticado**, redireccionar a la URL apropiada

```
dbManager.getUsers(criterio, function(users) {  
    if (users == null || users.length == 0) {  
        // No se ha autenticado  
    } else {  
        // Se ha autenticado  
    }  
});
```


Sesión y Autorización

- Una vez el usuario se autentica con éxito debemos **recordarlo**
 - El usuario con **email = J** está autenticado en el navegador X
- El objeto **sesión** es clave para identificar navegadores/clientes autenticados
- La sesión de express es un módulo externo **express-session**
 - <https://github.com/expressjs/session>

```
npm install express-session
```

- La función sesión se obtiene con **require**
 - La función puede recibir muchísimos parámetros de configuración opcionales. Algunos de los más comunes son:
 - **secret:** Es una clave secreta que se utiliza para firmar las cookies de sesión.
 - **resave:** (**true** / false) Indica si la sesión debe ser guardada de nuevo en el almacén de sesiones incluso **si no ha sido modificada** durante la solicitud.
 - **saveUninitialized:** (**true** / false) Indica si una sesión "no inicializada" debe ser guardada en el almacén de sesiones.

Sesión y Autorización

- La función sesión se integra con la aplicación con **app.use()**
- Ejemplo de configuración de sesión:

```
var app = express();
```

```
var expressSession = require('express-session');
```

```
app.use(expressSession({  
  secret: 'abcdefg',  
  resave: true,  
  saveUninitialized: true  
}));
```



La configuración de express-sesión se envía en un **objeto**
El objeto define: **secret**, **resave** y **saveUninitialized**

Sesión y Autorización

- La sesión es accesible desde todas las peticiones (request)
- Sus atributos se pueden leer/escribir mediante:
req.session.<clave del atributo>
- El usuario autenticado correctamente se almacenará en la sesión
 - Se debe guardar un valor que le identifique de forma única, ejemplo :
email

```
dbManager.getUser(criterio, function(users) {  
    if (users == null || users.length == 0) {  
        req.session.user = null;  
        // respuesta no autenticado  
    } else {  
        req.session.user = users[0].email;  
        // respuesta autenticado  
    }  
});
```

Sesión y Autorización

- Para eliminar de sesión un usuario podemos optar por:
 - Destruir la sesión ***req.session.destroy()***
 - Poner a null el atributo que identifica al usuario

```
app.get('/logout', function (req, res) {  
  req.session.user = null;  
  res.send("Usuario desconectado");  
})
```

Sesión y Autorización

- La **autorización** debe comprobar si el cliente tiene permiso para acceder a las URLs de la aplicación
- La aplicación puede consultar en todo momento si hay un usuario autenticado utilizando la sesión.
- Ejemplo:

```
app.get('/private', function (req, res) {  
  if ( req.session.user == null) {  
    // NO autenticado!  
    res.redirect("/login");  
    return;  
  }  
  ...  
})
```

Si **usuario == null** no hay
usuario autenticado
No puede entrar en **/privado**

- No es nada apropiado realizar el control de autorización en las funciones app.get/post
 - Mala arquitectura, replicación de código, dificultad para realizar modificaciones
 - Solo sirve para controlar URLs declaradas en app no directorios (como /public, etc.)

Enrutadores (express.Router())

- Los **enrutadores** se utilizan para **crear módulos de rutas independientes** en una aplicación Express.
- Permiten:
 - organizar las rutas en módulos.
 - separar la lógica de las rutas en archivos independientes.
 - reutilizar rutas en diferentes partes de la aplicación.
 - mantener el código más limpio y estructurado.
 - Los enrutadores se crean utilizando **express.Router()**.

Enrutadores (express.Router())

Definición de un router

```
const express = require("express");  
const router = express.Router();
```

```
router.get("/", function(req, res) {  
  res.send("Lista de usuarios");  
});
```

```
router.get("/:id", function(req, res) {  
  res.send("Usuario " + req.params.id);  
});
```

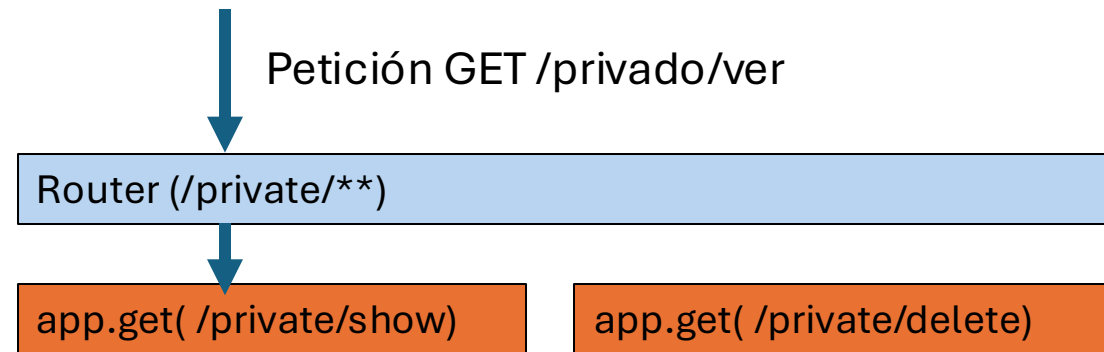
```
module.exports = router;
```

Uso del router en la aplicación

```
const usersRoutes = require("./routes/usersRoutes");  
app.use("/users", usersRoutes);
```

Enrutadores (express.Router())

- Los **enrutadores** permiten **organizar rutas y definir funciones (middleware)** que procesan las peticiones antes de generar la respuesta (interceptores).
- Si declaramos el uso de enrutador antes de agregar las URLs ***app.get/app.post*** procesará las peticiones antes que ellas



- En este caso, La función del enrutador puede:
 - Ejecutar cualquier lógica de negocio
 - **Dejar correr** la petición (para que la procese el siguiente elemento)
 - **Cortar la petición** (por ejemplo: redireccionándola)

Enrutadores (express.Router())

- Un router permite **funciones middleware** para procesar las peticiones.
- Con **.use(<func>)** se puede agregar una **función middleware** que se ejecutará cuando se procese una petición.
- La función middleware recibe normalmente tres parámetros
 - **req** → información de la petición
 - **res** → objeto para enviar la respuesta
 - **next** → función que pasa el control al siguiente middleware o ruta

• EjemploL

```
var routerAuthentication = express.Router();
routerAuthentication.use(function(req, res, next) {
    if ( req.session.user )
        // Hay usuario autenticado
        next();
    else
        // No hay usuario autenticado
        res.redirect("/login");
});
```

Si no hay usuario autenticado
redirecciona a /login

Si hay usuario autenticado
deja correr la petición

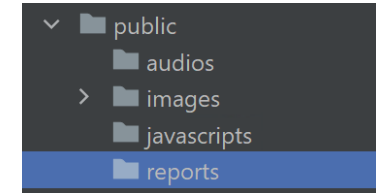
Enrutadores (express.Router())

- Una vez creado el enrutador se **agrega a la aplicación**
app.use(<ruta donde se aplica>, enrutador)
- Un mismo enrutador se puede aplicar en muchas rutas
- Por ejemplo:

```
app.use("/private/", routerAuthentication);  
app.use("/reports/", routerAuthentication);
```

```
app.use(express.static('public'));
```

```
app.get("/private/show", function(req, res)  
app.get("/private/delete", function(req, res)  
...
```



Agregar el enrutador a la aplicación en
/privado/
/informes/
**A todas las peticiones
GET , POST, ETC.**

Enrutadores (express.Router())

- En orden en que se agregan los enrutadores, directorios y respuestas (get/set) es crítico
- El orden determina quien responderá a la petición:
- La función **next()** de los routers deja continuar la petición

GET /private/show

```
① app.use("/private/", routerAuthentication);  
  app.use("/reports/", routerAuthentication);  
  
  app.use(express.static('public'));  
  
② app.get("/private/show", function(req, res)  
  app.get("/private/delete", function(req, res)  
  ...
```

Solo si el router ejecuta
next() pasará al siguiente

Enrutadores (express.Router())

- Sí el orden no es correcto a la petición será respondida por quién no pretendíamos
- MAL -> Ejemplo 1: petición get



GET /private/show

```
app.use(express.static('public'));
```

① `app.get("/private/show", function(req, res)`
`app.get("/private/delete", function(req, res)`

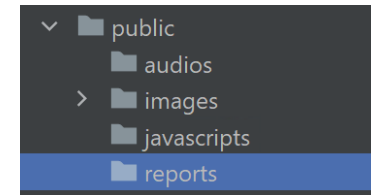
Responde a la petición
se termina el ciclo

```
app.use("/private/", routerAuthentication);  
app.use("/reports/", routerAuthentication);
```

Nunca se ejecuta

Enrutadores (express.Router())

- Sí el orden **no es correcto** a la petición será respondida por quien no pretendíamos
- MAL -> Ejemplo 2: acceso a un recurso



GET /reports/1.pdf

① `app.use(express.static('public'));`

```
app.get("/private/show", function(req, res)
app.get("/private/delete", function(req, res)
```

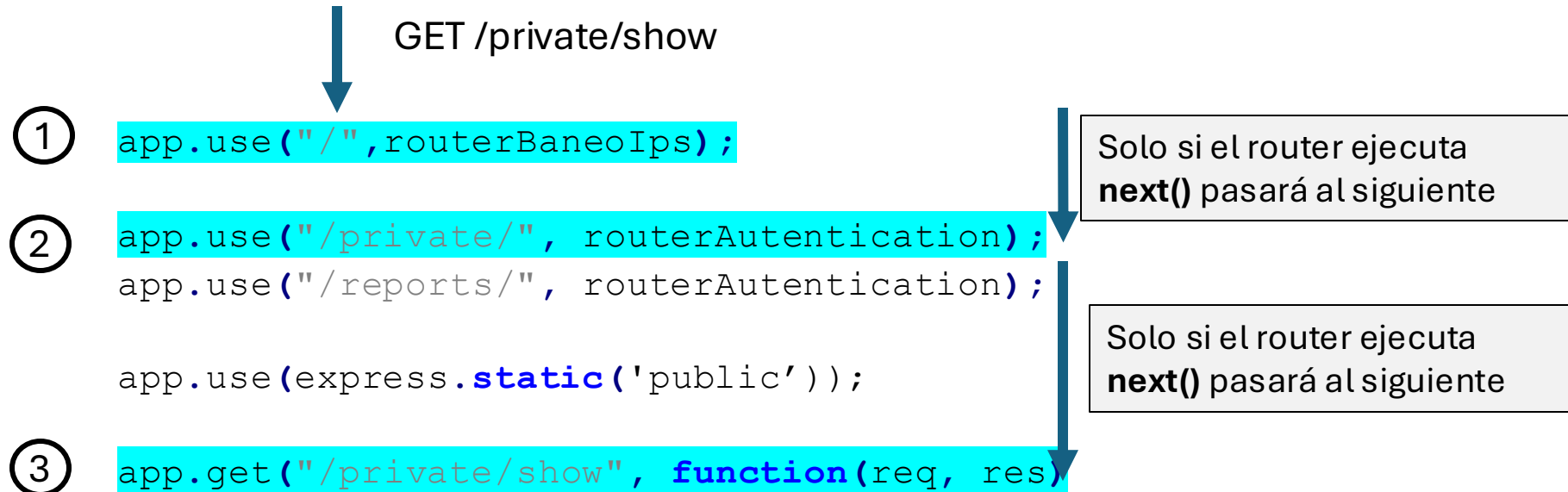
Responde a la petición
se termina el ciclo

```
app.use("/private/", routerAuthentication);
app.use("/reports/", routerAuthentication);
```

Nunca se ejecuta

Enrutadores (express.Router())

- Se pueden **aplicar múltiples enrutadores** sobre un mismo path
- Se delega una única acción en cada enrutador
- Por ejemplo:
 - routerAutentication -> comprueba si hay usuario en sesión
 - routerBaneoIPs -> comprueba si la IP está en una lista negra



Enrutadores (express.Router())

- Una buena recomendación de diseño es agrupar las URLs por **su nivel de autorización de forma jerárquica**
- Ejemplo de una aplicación que gestiona anuncios donde:
 - **Todos los usuarios** pueden **ver** y **reservar** anuncios
 - **Solo los propietarios del anuncio** pueden **modificar** y **eliminar**
- Podríamos usar las siguientes URLs:
 - `/user/ads/show/:id`
 - `/user/ads/reserve/:id`
 - `/user/owner/ads/update/:id`
 - `/user/owner/ads/delete/:id`
- Donde habría dos enrutadores
 - `routerUser` -> comprueba que hay un usuario en sesión
 - `routerOwner` -> comprueba que es el propietario del anuncio

Subida de ficheros



Subida de ficheros

- Se requiere el módulo externo ***express-fileupload***

```
npm install express-fileupload
```

- Se obtiene la función express-fileupload con require

```
var fileUpload = require('express-fileupload');
```

- Se agrega el objeto a la aplicación express (app.use())
 - La subida de ficheros ya estará disponible en la aplicación

```
app.use(fileUpload());
```

Subida de ficheros

- La petición (req) puede contener ficheros
 - Se accede a ellos con **req.files.<clave>**
 - Por ejemplo:
 - En un formulario que incluye el input de tipo **file** con clave **foto**
 - Debemos recordar el **enctype** de tipo **multipart/form-data**

```
<form method="POST" action="/savephoto" enctype="multipart/form-data">  
  <input type="file" name="photo" accept=".jpg" />  
  <input type="submit" value="Enviar" />  
</form>
```

- Como se procesa el fichero:
 - Se almacena en una **variable**
 - Se **copia en un directorio**
 - Elegimos el **directorio** y **nombre** del fichero
 - Podemos usar la función **file.mv(<directorio>,callback())**

Subida de ficheros

- Ejemplo de `file.mv(<directorio>,callback() )`

```
if (req.files.photo != null) {  
  var photo = req.files.photo;  
  photo.mv('public/photos/' + id + '.jpg', function(err) {  
    if (err) {  
      // ERROR  
    } else {  
      // EXITO  
    }  
  });  
}
```

Asegurarse de que no es **null**

Solo si hay error la variable **err**
tendrá un valor

Subida de ficheros

- El **path** donde se guarda el fichero es importante por ejemplo:
 - Directorio de acceso web **public/*** si queremos incluir la foto en la web
 - Directorios privados sí queremos que la petición pase por un controlador
- El **nombre** con el que se salva el fichero suele ser especificado por la lógica de negocio
 - Evita conflictos de nombres
 - Nombres o rutas que permitan asociar el fichero a un usuario asociar (por ejemplo: ID del usuario)
- En algunos casos es necesario hacer **comprobaciones de autorización** para acceder a ficheros de **directorio de acceso web**
 - **/static/reports** solo pueden acceder usuarios registrados.
 - **/static/photos/31** solo puede acceder el usuario 31

Manejo de errores



Captura de errores

- Por defecto y en fase de desarrollo se suele dejar que la aplicación propague errores
- La traza de error ofrece información útil para el desarrollador
- Por ejemplo: solicitud con un id mal formado no es ObjectId()
 - <http://localhost:8081/ads/RRRRR>
 - Al intentar formar un ObjectId(RRRR) produce un error

```
Error: Argument passed in must be a single String of 12 bytes or a string of 24 hex characters
    at new ObjectId (C:\Dev\workspaces\NodeApps\sdi-lab-node\node_modules\bson\lib\bson\objectid.js:57:11)
    at Function.ObjectId (C:\Dev\workspaces\NodeApps\sdi-lab-node\node_modules\bson\lib\bson\objectid.js:38:43)
```

- No debemos mostrar nunca esta información en producción
 - No es descriptiva para los usuarios
 - Potencialmente peligrosa, pueden detectar versiones de las tecnologías que utilizamos y buscar vulnerabilidades

Captura de errores

- Existen varios mecanismos para capturar los **errores en última instancia**
 - Cada función debería controlar todos sus errores, pero lograrlo con todos los posibles errores puede ser muy complejo
- Una forma global de capturar errores es incluir una función en la **aplicación que capture los errores controlados**
 - La incluimos con **app.use(función)** como elemento final de la aplicación
 - **Sí detecta un error/excepción no controlada muestra una respuesta genérica sin información técnica**

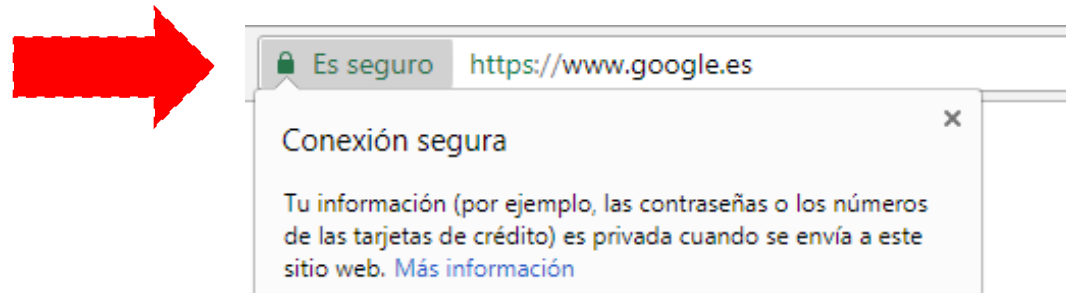
```
app.use( function (err, req, res, next ) {  
    console.log("Error producido: " + err);  
    if (! res.headersSent) {  
        res.send("Recurso no disponible");  
    }  
});
```

← Función de manejo de errores

```
app.listen(app.get('port'), function() {
```

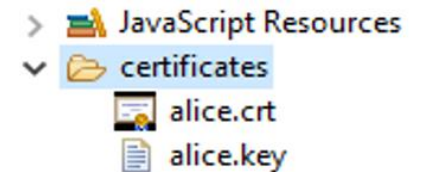
Https

- **Https** es un protocolo de transferencia **seguro** para hipertexto basado en http.
- Cifra un canal de comunicación entre el servidor y navegador utilizando certificados SSL/TLS
 - Sí los datos son interceptados en ese canal, estos estarán cifrados.
- Los navegadores dan información específica si una web usa https:
 - Datos del certificado usado para cifrar (quien lo ha emitido)
- Cualquiera puede emitir un certificado, pero hay varias autoridades certificadoras confiables.



Https

- Para agregar cifrado http incluimos los certificados en **una carpeta privada**
 - certificado.crt – certificado
 - certificado.key – clav
- Incluimos los módulos **https** y **fs** (filesystem) para procesamiento de ficheros



```
var fs = require('fs');  
var https = require('https');
```

- Modificamos la creación del canal **http** por -> **https**
 - Además del **listen** se debe incluir un **createServer()** que indica donde esta los certificados

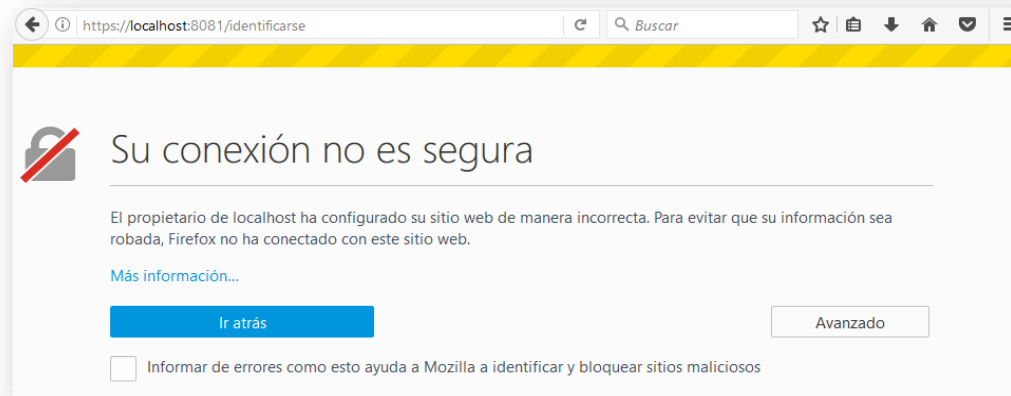
```
https.createServer({  
  key: fs.readFileSync('certificates/alice.key'),  
  cert: fs.readFileSync('certificates/alice.crt')  
}, app).listen(app.get('port'), function() {  
  console.log("Servidor activo");  
});
```

Https

- La aplicación ya usa https, las comunicaciones están cifradas.
- Aunque el certificado no está emitido por una entidad confiable (lo hemos generado nosotros):
 - Nuestro navegador nos lo hará saber:



- Probablemente debamos agregar la página a excepciones de seguridad





Escuela de Ingeniería Informática

Escuela de Ingeniería Informática
School of Computer Science Engineering

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Sistemas Distribuidos e Internet

Parte 2

Introducción a Node.js

Desarrollo de aplicaciones y servicios web
modernos con JavaScript



Dr. Edward Rolando Núñez Valdez

nunezedward@uniovi.es